EST-744

Fundamentos de Redes Neuronales

Juan Zamora O.

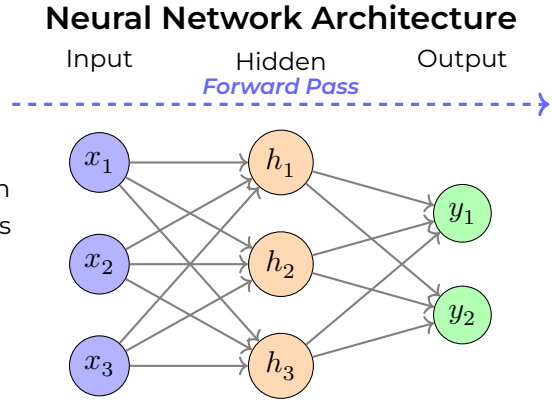
Deep Learning

What is a Neural Network?

A neural network is a system of interconnected “neurons” that learns to map inputs to outputs

Biological inspiration: mimics how neurons in the brain process information

Practical reality: mathematical functions composed of simpler functions



Why Neural Networks?

- Traditional ML: features carefully engineered by humans
- Neural Networks: **automatically learn** useful representations

Analogy: - A chef (human) vs. a kitchen robot (neural network) - Chef: manually prepares ingredients - Robot: learns what cuts and prep work matter

A Single Neuron

$$y = \sigma(w \cdot x + b)$$

- **Input** x: features (image pixels, word vectors, etc.)
- **Weights** w: learnable parameters (what matters)
- **Bias** b: threshold adjustment
- **Activation** $\sigma()$: non-linearity (ReLU, sigmoid, etc.)

Key insight: Without activation functions, stacking neurons = one linear transformation (useless for complex patterns)

Activation Functions

Function	Formula	When to use
ReLU	$\max(0, x)$	Hidden layers (most common)
Sigmoid	$\frac{1}{1+e^{-x}}$	Binary classification output
Tanh	$\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	RNNs, hidden layers

Intuition: Activation functions add non-linearity, allowing networks to learn curves and decision boundaries

Layers and Forward Pass

```
Input Layer → Hidden Layer 1 → Hidden Layer 2 → Output Layer
   [3 neurons]    [5 neurons]    [4 neurons]     [2 neurons]
```

Each layer: 1. Takes input from previous layer 2. Applies weights + bias 3. Applies activation function 4. Passes output to next layer

Forward pass: input flows through all layers to produce prediction

Depth and Width

- **Depth**: number of layers (deep = many layers)
- **Width**: neurons per layer

Trade-off: - More layers = can learn more complex patterns - Too many layers = harder to train, risk of vanishing gradients

For this course: start with 2-3 hidden layers unless specific needs

Loss Function

A **loss function** measures how wrong the network is:

$$L = \text{mean}((y_{true} - y_{pred})^2)$$

- **MSE Loss**: regression problems (continuous output)
- **Cross-entropy Loss**: classification problems (probabilities)

Goal during training: minimize loss by adjusting weights

Gradient Descent

Intuition: Imagine a ball rolling down a hill (loss surface) - Calculate the slope (gradient) - Roll down the steepest direction - Repeat until you reach the valley

$$w_{new} = w_{old} - \alpha \cdot \frac{dL}{dw}$$

- α = learning rate (step size)
- $\frac{dL}{dw}$ = gradient (slope of loss w.r.t. weight)

How Networks Learn

Backpropagation: Efficiently compute gradients for all weights

1. **Forward pass**: compute predictions and loss
2. **Backward pass**: compute gradients layer-by-layer (chain rule)
3. **Update**: adjust weights using gradients

Why backward? - Start from loss (what we care about) - Work backward to earlier layers - Reuse intermediate calculations (efficient)

Optimization: SGD to Adam

Optimizer	Idea	Learning Rate
SGD	Simple gradient descent	Fixed (manual tuning)
Momentum	Accumulate direction	Fixed
Adam	Adaptive per-parameter	Auto-adjusting

For this course: Use Adam (robust, requires less tuning)

A Simple Classification Task

Goal: Predict if a handwritten digit is 0-9

Data: MNIST (28×28 pixel images)

Network: - Input: 784 pixels (28 × 28 flattened) - Hidden 1: 128 neurons, ReLU - Hidden 2: 64 neurons, ReLU - Output: 10 neurons, softmax (probabilities for digits 0-9)

Code with Keras/TensorFlow

```
import tensorflow as tf
from tensorflow import keras

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = x_train.reshape(-1, 784).astype('float32') / 255.0
x_test = x_test.reshape(-1, 784).astype('float32') / 255.0

# Build network
model = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(784,)),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])

# Compile (loss + optimizer)
model.compile(
```

Monitoring Training

```
# Evaluate on test set
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test accuracy: {test_acc:.4f}")

# Make predictions
predictions = model.predict(x_test[:5])
print(predictions)  # probabilities for each digit
```

Typical results: ~97-98% accuracy with this simple architecture

What to visualize: - Training loss curve (should decrease) - Accuracy improvement over epochs - Predictions on a few test examples

What We've Covered

- Neurons as basic units (weights, bias, activation)
- Layers and forward passes
- Loss functions and gradient descent
- Backpropagation (the learning algorithm)
- Optimizers (Adam as default choice)
- Full training pipeline in code

Common Pitfalls

- **Learning rate too high**: weights oscillate, don't converge
- **Too many layers**: vanishing gradients, hard to train
- **Imbalanced data**: accuracy misleading
- **Overfitting**: memorizes training data, fails on test data

Hands-On Tasks

1. **Modify architecture**: Try different numbers of layers/neurons. How does it affect accuracy and training speed?
2. **Change activation functions**: Replace ReLU with tanh. Compare results.
3. **Experiment with learning rates**: Train with lr=0.001 and lr=0.1. What happens?
4. **Regularization**: Add `keras.layers.Dropout(0.2)` between hidden layers. Does overfitting decrease?
5. **Visualize weights**: Print `model.get_weights()`. Do they look reasonable?

From Networks to Text

Neural networks work on **any numeric input**—but how do we feed text?

Class 2: Embeddings - Convert words to vectors - Learn meaning from context - Visualize semantic relationships

“The quick brown fox” → Vector representations that capture “meaning”

Questions & Discussion

Thank you!

Key takeaway: Neural networks are function approximators that learn through gradient-based optimization. Everything else is architecture and tricks.

Next: How do we feed text to these networks?